Task 1. Provision a Mumbai EC2 instance

For a reusable configuration, add variables to your existing Terraform project.

`variables.tf`

```
None
variable "instance_type" {
  type    = string
  default = "t2.micro"
}

variable "instance_name" {
  type    = string
  default = "MusicApp-Server"
}
```

Update the EC2 resource:

```
None
resource "aws_instance" "music_app" {
  ami           = data.aws_ami.linux.id
  instance_type = var.instance_type

  vpc_security_group_ids = [aws_security_group.web.id]

  tags = {
    Name = var.instance_name
  }
}
```

The provider region must be `ap-south-1`.

Task 2. Allow HTTP and SSH traffic

Add this resource, assuming the default VPC exists:

None

```
data "aws_vpc" "default" {
  default = true
}

variable "ssh_cidr" {
  description = "Trusted client IPv4 CIDR for SSH"
  type        = string
}

resource "aws_security_group" "web" {
  name        = "music-app-web-sg"
  description = "Allow HTTP and restricted SSH"
  vpc_id      = data.aws_vpc.default.id

  ingress {
    description = "HTTP"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "SSH from trusted IP"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = [var.ssh_cidr]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
```

```
  tags = {
    Name = "music-app-web-sg"
  }
}
```

Create `terraform.tfvars`:

```
None
ssh_cidr = "YOUR.PUBLIC.IP.ADDRESS/32"
```

The example allows HTTP from any IPv4 address but limits SSH to your trusted IP. If the Node.js application listens on port 3000, configure a reverse proxy such as Nginx to expose it on port 80, or add a separate restricted rule for port 3000.

Task 3. Use a variable for the EC2 instance name

Change the tag to:

```
None
tags = {
  Name = var.instance_name
}
```

Now the instance name can be changed in `terraform.tfvars` without editing the EC2 resource:

```
None
instance_name = "MusicApp-Server"
instance_type = "t2.micro"
ssh_cidr      = "YOUR.PUBLIC.IP.ADDRESS/32"
```

Task 4. Create `deploy.sh`

```
None
#!/usr/bin/env bash
```

```
set -euo pipefail

terraform init
terraform fmt -check
terraform validate
terraform plan -out=tfplan
terraform apply tfplan

echo "Deployment completed."
echo "EC2 public IP: $(terraform output -raw public_ip)"
```

Add the matching output to Terraform:

```
None
output "public_ip" {
  description = "Public IPv4 address of the EC2 instance"
  value       = aws_instance.music_app.public_ip
}
```

Run the script on Linux or WSL:

```
None
chmod +x deploy.sh
./deploy.sh
```

Explanation: `set -euo pipefail` makes the script stop on errors and unset variables. Saving and applying the plan ensures that the reviewed plan is the one applied.

Task 5. Review and improve the deployment scripts

Original approach	Recommended improvement	Benefit
SSH open to everyone	Restrict SSH to a trusted CIDR or use Systems Manager	Reduces attack surface

Credentials in code	Use environment variables, IAM roles, or a secret manager	Protects secrets
`terraform apply -auto-approve` everywhere	Review plans and require approval for production	Reduces accidental changes
Image tag `latest`	Use an immutable commit-SHA tag	Improves traceability and rollback
No validation	Run `terraform fmt`, `validate`, and `plan` before applying	Catches errors early